

Inteligencia Artificial



Inteligencia Artificial

Estrategias de búsqueda informada

Referencias:

https://www.youtube.com/watch?v=UVw_sX7AMG0

https://www.youtube.com/watch?v=yxN6yR_7yJM&t=266s

https://www.youtube.com/watch?v=hPB9plUYu_Q

<https://www.youtube.com/watch?v=1gszEk8rUS4>

REINVENTEMOS
EL FUTURO



Estrategias de búsqueda informada

En el campo de la inteligencia artificial, las estrategias de búsqueda informada son importantes para la resolución de problemas complejos, de manera eficiente.

A diferencia de las búsquedas no informadas, las búsquedas informadas, utilizan el conocimiento específico del dominio del problema para guiar la exploración del espacio de búsqueda, reduciendo el tiempo y los recursos necesarios para encontrar una solución óptima.

En este contexto se estudiará la **búsqueda voraz**; el ampliamente utilizado **algoritmo A***, conocido por su capacidad de encontrar la ruta más corta garantizada; y finalmente se revisarán las técnicas para la **optimización de búsqueda en grafos**.

Búsqueda voraz

- Un método de búsqueda informado utiliza el conocimiento del dominio relacionado con la ubicación del estado objetivo y podría ayudar a alcanzar el estado objetivo más rápidamente que los métodos de búsqueda no informados. El conocimiento del dominio se representa mediante una función heurística, usualmente denotada como $h(n)$, que es igual al costo estimado de la ruta más barata o económica, desde el estado en el nodo n hasta el estado objetivo, también $h(n)$ puede ser la distancia en línea recta, o euclidiana (Chugani, 2024), desde cada nodo hasta el objetivo.
- Una función heurística es una función utilizada en algoritmos de búsqueda para estimar o aproximar el costo, o la distancia, de un estado dado al estado objetivo. La idea es proporcionar una estimación informada que guíe al algoritmo, al priorizar la exploración de caminos que parecen más prometedores para alcanzar la solución.

Búsqueda voraz

- La **búsqueda voraz** conocida también como **Greedy Best-First Search**, conocida también como búsqueda avariciosa, prioriza la exploración de nodos que parecen más cercanos al objetivo basándose en una heurística.
- Aunque puede ser una búsqueda rápida, su naturaleza "**avariciosa**" significa que no necesariamente es óptima en cuanto a costos, esto se debe a que el enfoque ambicioso no mira hacia adelante y solo considera el paso actual.
- En este tipo de búsqueda se expande primero el nodo con el **$h(n)$** más "**barato**", ya que parece estar más cerca del estado objetivo.
- **Greedy Best-First Search**, es un paradigma algorítmico que construye una solución **pieza por pieza** , eligiendo siempre la siguiente nodo que ofrece el **beneficio más obvio** e inmediato . Los algoritmos voraces se utilizan para problemas de optimización.

Ventajas de los algoritmos voraces

- Sencillo y fácil de entender, suelen ser fáciles de implementar y razonar.
- Eficientes para ciertos problemas, como encontrar la ruta más corta en un gráfico con pesos de arista no negativos.
- Tiempo de ejecución rápido, tienen una complejidad temporal menor en comparación con otros algoritmos para ciertos problemas.
- Intuitivo y fácil de explicar, el proceso de toma de decisiones en un algoritmo codicioso suele ser fácil de entender y justificar.
- Los algoritmos voraces se pueden combinar con otras técnicas para diseñar algoritmos más sofisticados para problemas desafiantes.

Desventajas de los algoritmos voraces

- Los algoritmos codiciosos priorizan los óptimos locales sobre los óptimos globales, lo que conduce a soluciones subóptimas en algunos casos.
- Demostrar la optimalidad de un algoritmo voraz puede ser un desafío y requerir un análisis cuidadoso.
- Sensibilidad al orden de entrada, el orden de los datos de entrada puede afectar la solución generada por un algoritmo codicioso.
- Aplicabilidad limitada, los algoritmos codiciosos no son adecuados para todos los problemas y pueden no ser aplicables a problemas con restricciones complejas.

Búsqueda voraz

En la **búsqueda voraz**, el algoritmo siempre elige la opción que parece ser la mejor *inmediatamente*, sin pensar mucho en las consecuencias a largo plazo. En este contexto **"Best-First"** (primero el mejor), prioriza explorar los nodos que parecen estar más cerca del objetivo, según una función llamada **"heurística"**.

Puntos clave para explicarle a tus alumnos sobre este código:

1. **La Cola de Prioridad (FRONTERA):** Es el corazón del algoritmo. A diferencia de BFS (que usa una cola normal FIFO) o DFS (que usa una Pila LIFO), la Búsqueda Voraz reordena constantemente sus opciones. Siempre empuja hacia adelante al vecino que tenga el $h(n)$ más bajo.
2. **La Heurística $h(n)$:** Es una "suposición educada". Por ejemplo, si buscamos una ciudad en un mapa, $h(n)$ podría ser la distancia en línea recta (vuelo de pájaro) hacia el objetivo.
3. **El defecto "Voraz":** en ningún lado del código se suma el costo del camino recorrido (el esfuerzo que ya costó llegar ahí). Solo mira hacia adelante ($h(n)$). Por eso puede tomar decisiones que parecen buenas a corto plazo, pero que terminan metiéndolo en un callejón sin salida (como caminar hacia un objetivo visible, pero chocar con un muro gigante que obliga a rodear todo).

Algoritmo – Búsqueda voraz

Este algoritmo prioriza la cercanía aparente, pero sacrifica la garantía de optimalidad.

```
Algoritmo Busqueda_Voraz_Best_First(nodo_inicial, nodo_objetivo):

    // 1. Inicialización
    Crear una Cola_de_Prioridad llamada FRONTERA // Ordenada por el valor de la heurística h(n)
    Crear un conjunto vacío llamado EXPLORADOS // Para recordar por dónde ya pasamos

    // Insertamos el primer nodo calculando su distancia estimada a la meta
    Insertar nodo_inicial en FRONTERA con prioridad h(nodo_inicial)

    // 2. Ciclo principal de búsqueda
    MIENTRAS FRONTERA no esté vacía HACER:

        // El algoritmo "voraz" siempre saca el nodo con el MENOR valor de h(n)
        nodo_actual = Extraer_mejor_nodo(FRONTERA)

        // 3. Condición de éxito
        SI nodo_actual ES IGUAL A nodo_objetivo ENTONCES:
            RETORNAR "¡Éxito! Solución encontrada" (y reconstruir el camino)
        FIN SI

        // Lo marcamos como visitado para no volver a él y evitar ciclos
        Añadir nodo_actual a EXPLORADOS

        // 4. Expansión de vecinos (mirar el siguiente paso)
        PARA CADA vecino DE nodo_actual HACER:

            SI vecino NO ESTÁ EN EXPLORADOS Y vecino NO ESTÁ EN FRONTERA ENTONCES:
                // Evaluamos qué tan bueno "parece" este vecino usando la heurística
                prioridad = h(vecino)

                // Lo metemos a la lista de opciones
                Insertar vecino en FRONTERA con la prioridad calculada
            FIN SI

        FIN PARA

    FIN MIENTRAS

    // 5. Si la frontera se vacía y nunca vimos el objetivo
    RETORNAR "Fallo: No se encontró un camino"

Fin Algoritmo
```

Algoritmo – Búsqueda voraz –ejemplo

1. El turista guiado por un edificio alto (Navegación Visual)

- **El contexto:** Imagina que eres un turista en París, no tienes Google Maps, pero ves a lo lejos la Torre Eiffel (tu objetivo).
- **Cómo aplica la Búsqueda Voraz:** En cada intersección, miras hacia la Torre y tomas la calle que apunte **más directamente** hacia ella. Tu "heurística" $h(n)$ es la distancia visual en línea recta. No te importa nada más que acercarte visualmente.
- **La trampa (El problema de ser voraz):** Al seguir siempre la calle que apunta a la Torre, podrías terminar metiéndote en un callejón sin salida, o chocar contra el río Sena justo donde no hay un puente. Tomaste la mejor decisión inmediata en cada esquina, pero ignoraste la estructura real de la ciudad, obligándote a dar una vuelta gigante al final.

Algoritmo – Búsqueda voraz –ejemplo

2. El cajero dando el vuelto (Algoritmo de Monedas)

El contexto: Un cajero automático o un vendedor tiene que darte \$0.85 de cambio y quiere darte la menor cantidad de monedas posibles.

Cómo aplica la Búsqueda Voraz: El cajero actúa de forma voraz eligiendo siempre **la moneda más grande que no se pase del total**. Primero toma una moneda de \$0.50 (queda \$0.35). Luego toma la más grande disponible, una de \$0.25 (queda \$0.10). Finalmente toma una de \$0.10. ¡Solucionado rápidamente!

La trampa (El problema de ser voraz): Funciona bien con nuestro sistema de dinero. Pero imagina un sistema inventado con monedas de 1, 3 y 4 centavos, y hay que dar un vuelto de 6 centavos. El algoritmo voraz agarrará la de 4 primero (quedan 2), y luego dos de 1. Entregó **3 monedas** (4, 1, 1). Si no hubiera sido voraz, habría visto que la solución óptima a largo plazo era entregar **2 monedas** (3, 3).

Algoritmo – Búsqueda voraz –ejemplo

3. Aceptar el primer trabajo con el sueldo más alto (Decisiones de Vida)

El contexto: Un recién graduado tiene varias ofertas de empleo y quiere maximizar su riqueza a lo largo de su carrera.

Cómo aplica la Búsqueda Voraz: Al ver las opciones, elige inmediatamente el trabajo A porque le pagan \$100 dólares más al mes que el trabajo B. Su heurística es el salario inicial inmediato.

La trampa (El problema de ser voraz): Cayó en lo que en Inteligencia Artificial se llama un "**Máximo Local**". El trabajo A pagaba un poco más hoy, pero era en una empresa sin oportunidades de ascenso. El trabajo B pagaba menos al principio, pero ofrecía capacitación y subir a un puesto gerencial en un año. Por no mirar a largo plazo (el costo acumulado), tomó una decisión subóptima.

Algoritmo de Búsqueda A*

Considera no solo la distancia a cada nodo del objetivo, sino también el costo de llegar a cada nodo. A* es el algoritmo de búsqueda informado más común. El conocimiento de dominio se representa mediante una función $f(n)$, que evalúa tres componentes para guiar el proceso de búsqueda:

$$f(n) = g(n) + h(n)$$

$g(n)$ = costo real de llegar al nodo n desde el nodo de estado inicial

$h(n)$ = costo estimado de la ruta más barata desde el estado en el nodo n hasta el estado objetivo
= distancia en línea recta desde cada nodo hasta la meta.

Algoritmo de Búsqueda A*

El algoritmo A*, se implementa para la búsqueda de caminos orientada a encontrar la ruta de menor costo desde un nodo inicial a un nodo objetivo. Se diferencia de otros algoritmos de búsqueda informada porque utiliza una función heurística para estimar el costo restante para llegar al objetivo. Las heurísticas admisibles más comunes son:

- Distancia Euclidiana: distancia en línea recta, para problemas de navegación.
- Distancia Manhattan: suma de diferencias absolutas en las coordenadas, útil en entornos cuadriculados.
- Distancia de Hamming: número de posiciones en las que dos cadenas son diferentes, útil en problemas de búsqueda de cadenas.

Ventajas del algoritmo de Búsqueda A*

- Eficiencia, generalmente es más rápido que otros algoritmos de búsqueda no informada, especialmente en problemas grandes.
- Optimalidad, garantiza la solución óptima si la heurística es admisible.
- Adaptabilidad, se puede adaptar a una amplia variedad de problemas simplemente cambiando la función heurística.

Desventajas del algoritmo de Búsqueda A*

- Puede requerir una gran cantidad de memoria, para almacenar la lista de nodos que han sido generados, pero que aún no han sido explorados.
- La elección de una buena heurística puede resultar compleja, y realmente importa para el rendimiento del algoritmo. Una heurística mal elegida puede resultar en un rendimiento pobre.

Algoritmo – A* -ejemplo

1. Inteligencia Artificial en Videojuegos (El movimiento de los personajes)

El contexto: En juegos como *Age of Empires*, *The Sims* o *League of Legends*, le das clic a un personaje para que camine hacia un bosque, pero en medio hay muros, ríos y otros edificios.

Cómo aplica A*: El juego divide el mapa en una cuadrícula. El algoritmo calcula $g(n)$ (cuántos cuadros de energía/tiempo ya caminó el personaje) y le suma $h(n)$ (la distancia en línea recta hacia el punto del clic).

Por qué usa A*: Si usara Búsqueda Voraz, el personaje caminaría directo hacia el objetivo, chocaría contra un muro curvo y se quedaría atascado. Si usara UCS (Dijkstra), el juego se congelaría por *lag* porque el algoritmo calcularía caminos hacia las espaldas del personaje antes de avanzar. A* logra esquivar el muro de forma natural y rápida, yendo directo a la meta sin atascarse.

Algoritmo – A* -ejemplo

2. Algoritmos de enrutamiento GPS de Larga Distancia

El contexto: Estás en Madrid y pones en el GPS que quieres ir a Barcelona.

Cómo aplica A:* El algoritmo evalúa las carreteras. Suma el costo real de los peajes y el tráfico de los kilómetros ya recorridos ($g(n)$) con la distancia en línea recta que aún le falta para llegar a Barcelona ($h(n)$).

Por qué usa A:* UCS (Dijkstra) buscaría la ruta más corta expandiendo un círculo perfecto alrededor de Madrid; eso significa que perdería tiempo evaluando carreteras que van hacia Portugal (¡en dirección contraria!). A*, gracias a su heurística ($h(n)$), "estira" la búsqueda hacia el este (hacia Barcelona), ignorando las carreteras del oeste, ahorrando una cantidad masiva de cálculos computacionales pero garantizando la ruta más rápida.

Algoritmo – A* -ejemplo

3. Robots Logísticos en los Almacenes (Ej. Robots Kiva de Amazon)

El contexto: Miles de pequeños robots naranjas se mueven a toda velocidad por los gigantescos almacenes de Amazon llevando estanterías, esquivándose unos a otros y sin chocar.

Cómo aplica A*: Cada robot necesita ir desde su posición hasta una estación de empaque. El sistema central utiliza A* (y sus variantes) para trazar la ruta. Calcula el consumo de batería y tiempo real ya gastado ($g(n)$) más la distancia estimada a la estación ($h(n)$).

Por qué usa A*: Si un robot va en línea recta pero detecta que hay un "atasco" de otros tres robots en el pasillo, A* suma ese alto costo ($g(n)$) y se da cuenta instantáneamente de que rodear el pasillo por una ruta ligeramente más larga ($h(n)$) resulta en un costo total ($f(n)$) menor. Toma la decisión óptima sin perder tiempo.



Fin de la presentación